Luxo Jr. — přímá a inverzní kinematika

Technická dokumentace • stav kalibrace 30. 10. 2026 • jednotky: mm, stupně

Dokument popisuje matematiku skutečné poslední verze kinematics.py. Appendix A obsahuje kompletní opravenou implementaci v Pythonu (včetně záporného znaménka serva 5).

1. Souřadnice, parametry a kalibrace

Globální soustava: stůl z = 0, osa Z míří vzhůru, základna v (0,0,0). V referenční poloze horní rameno směřuje do záporného X. Rotace základny působí kolem osy Z.

q = (q1,q2,q3,q4,q5) jsou úhly serv ve stupních. Rovina ramen je v lokálních souřadnicích XZ; servo 1 ji otáčí do prostoru.

```
H = 100; L1 = 230; L2 = 175; L3 = 85; d = 55; b = 75
```

```
limity q: [0,180], [95,140], [60,133], [45,113], [0,180]
```

```
q_ref = (90, 114, 97, 57, 97)
```

L3 je stále vodorovný mezikus; d je excentrické odsazení osy stínidla od osy serva 4; b je vzdálenost od referenčního bodu osy stínidla ke svíticímu elementu. Rozměry d a b jsou přibližné.

2. Převod serv na geometrické úhly

```
psi   = -(q1 - 90) * pi/180
alpha = (90 + q2 - 114) * pi/180
beta  = (180 + q3 - 97) * pi/180
theta = -1.5 * (q4 - 57) * pi/180
phi   = -(9/7) * (q5 - 97) * pi/180
```

Úhly alpha a beta jsou absolutní a vzájemně nezávislé (paralelogram). Theta a phi nezávisí na sklonu ramen. Znaménko phi odpovídá poslední ručně ověřené opravě.

3. Přímá kinematika (forward kinematics)

```
Rz(psi) = [[cos(psi), -sin(psi), 0],
           [sin(psi),  cos(psi), 0],
           [0,         0,        1]]
```

```
O = (0,0,0)
S = (0,0,H)
E = S + Rz(psi) * (L1*cos(alpha), 0, L1*sin(alpha))
U = E + Rz(psi) * (L2*cos(beta), 0, L2*sin(beta))
A4 = U + Rz(psi) * (-L3, 0, 0)
```

O je základna, S rameno/ramenní kloub, E loket, U konec horního ramene, A4 osa serva 4.

```
v_local = (sin(theta)*cos(phi), sin(phi), -cos(theta)*cos(phi))
D = Rz(psi) * v_local  # jednotkový vektor paprsku
A5 = A4 + Rz(psi) * (-d*cos(theta), 0, d*sin(theta))
B = A5 + b*D          # poloha svíticího elementu
```

Poznámka: A5 je geometrický referenční bod na ose stínidla, nikoli tvrzení o fyzickém průsečíku os serv. Současný model předpokládá, že servo 5 mění směr paprsku, ale neposouvá bod A5. Je to kalibrační aproximace.

4. Průsečík světelného paprsku se stolem

```
P(t) = B + t*D
z_stolu = 0 => t = -Bz/Dz
X = Bx - Bz*Dx/Dz
Y = By - Bz*Dy/Dz
```

Průsečík existuje jako dopad světla jen při Bz > 0, Dz < 0 a t >= 0. Pro vodorovný či vzhůru mířící paprsek vrací implementace None.

```
gamma(q) = arccos(clip(-Dz,-1,1)) * 180/pi
```

Gamma je úhel paprsku od svislého směru dolů.

5. Inverzní kinematika (inverse kinematics)

Vstupem je cílový bod T = (Xt,Yt), požadovaný úhel gamma_t od svislice a výchozí poloha q_start. Výstupem je q* v mechanických limitech. Úloha má obvykle více řešení, protože požadujeme pouze dvě souřadnice a velikost náklonu, nikoliv azimut náklonu.

```
q* = argmin_{q_min <= q <= q_max} ||r(q)||^2
```

Vlastní implementace používá scipy.optimize.least_squares a reziduální vektor:

```
dz_safe = min(Dz, -0.05)
t_safe  = max(0, -Bz/dz_safe)
P_safe  = B + t_safe*D
r_xy    = 8*(P_safe_xy - T)/100
r_tilt  = (gamma(q) - gamma_t)/15
r_up    = 15*max(0, Dz + 0.05)
r_below = max(0, -Bz)/100
r_move  = 0.12*(q - q_start)/100
r(q)    = concat(r_xy, r_tilt, r_up, r_below, r_move)
```

Pro hledání se zkouší výchozí poloha, referenční poloha a 10 reprodukovatelných náhodných počátků. Výsledky se porovnávají podle skutečné chyby dopadu, chyby náklonu a malého zvýhodnění menšího pohybu:

```
score = error_mm + 5*tilt_error_deg
        + 0.5*norm((q - q_start)/180)
```

Výstup inverse: (q, error_mm, tilt_error_deg, hit_or_None). Číslo error_mm je chyba souřadnic dopadu v modelu; není to měření skutečné lampy. Funkce sama nepřisuzuje řešení stav „dosažitelné“: volající musí zkontrolovat obě chyby a existenci dopadu.

6. Referenční kontrola

```
q_ref = (90,114,97,57,97)
alpha=90°, beta=180°, theta=0°, phi=0°, psi=0°
A4 = (-260,0,330) mm
A5 = (-315,0,330) mm
B  = (-315,0,255) mm
D  = (0,0,-1)
Hit = (-315,0,0) mm; gamma = 0°
```

Skutečně pozorovaný dopad byl přibližně X = -300 mm; rozdíl 15 mm odpovídá nejistotě rozměrů. Referenční poloha a další cíle byly vizuálně ověřeny na skutečné lampě, nikoli laboratorně změřeny.

7. Omezení modelu a modularita

Převody serv 2 a 3 jsou zatím přibližné. Zjednodušena je i mechanika excentrického stínidla při současném naklopení serv 4 a 5. Zadaný úhel gamma nefixuje azimut směru paprsku. Výsledky IK mohou končit na limitech a nemusí být dosažitelné. Kinematická knihovna neposílá příkazy do zařízení; hardware ESP8266, simulace a budoucí gesta mají zůstat v oddělených modulech.

Appendix A — úplná implementace kinematics.py

Níže je úplný kód z posledního projektu se zapracovanou poslední potvrzenou změnou: shade_side_gain = -9.0/7.0. Ostatní výpočty odpovídají poslednímu uloženému souboru.

```
"""Luxo Jr. calibrated *provisional* 5-axis model. Units: mm, degrees.

Servo 2 and 3 set ABSOLUTE arm angles independently (parallelogram linkage).

Servo 4 and 5 set shade direction independently of both arms.

Servo 1 rotates the entire assembly about the vertical Z axis.

No hardware communication.

"""

from dataclasses import dataclass

import numpy as np

from scipy.optimize import least_squares

DEG = np.pi / 180.0

LIMITS = np.array([[0, 180], [95, 140], [60, 133], [45, 113], [0, 180]], dtype=float)
```

```
REFERENCE = np.array([90., 114., 97., 57., 97.])

@dataclass(frozen=True)
class Calibration:
    lower_length: float = 230.0
    upper_length: float = 175.0
    shoulder_height: float = 100.0
    horizontal_extension: float = 85.0  # always horizontal, points left at reference
    shade_axis_offset: float = 55.0  # axis of bulb is left of servo 4 at reference
    bulb_from_axis_5: float = 75.0  # approximate along light direction
    lower_vertical_servo: float = 114.0
    lower_gain: float = 1.0  # provisional
    upper_horizontal_servo: float = 97.0
    upper_gain: float = 1.0  # provisional
    shade_pitch_zero: float = 57.0
    shade_pitch_gain: float = -1.5  # verified sign: 57 -> 87 raises light direction
    shade_side_zero: float = 97.0
    shade_side_gain: float = -9.0 / 7.0  # corrected sign, checked on physical lamp
    base_zero: float = 90.0
    base_gain: float = -1.0  # verified: 0 counterclockwise, 180 clockwise, top view

CAL = Calibration()


def _rz(angle):
    c, s = np.cos(angle), np.sin(angle)
    return np.array([[c, -s, 0.], [s, c, 0.], [0., 0., 1.]])


def forward(q, cal=CAL):
    """Return (7x3 points, unit ray): base, shoulder, elbow, upper_end, axis4, shade_axis, bulb.

    Local Y is defined by the model; physical left/right is calibrated.
    The wrist offset direction and bulb offset are still provisional.
    """
    q = np.asarray(q, dtype=float)
    if q.shape != (5,):
        raise ValueError('Expected five servo angles')
    yaw = (q[0] - cal.base_zero) * cal.base_gain * DEG
    lower = (90.0 + (q[1] - cal.lower_vertical_servo) * cal.lower_gain) * DEG
    upper = (180.0 + (q[2] - cal.upper_horizontal_servo) * cal.upper_gain) * DEG
    pitch = (q[3] - cal.shade_pitch_zero) * cal.shade_pitch_gain * DEG
    side = (q[4] - cal.shade_side_zero) * cal.shade_side_gain * DEG
    R = _rz(yaw)
    origin = np.zeros(3)
    shoulder = np.array([0., 0., cal.shoulder_height])
    elbow = shoulder + R @ np.array([cal.lower_length * np.cos(lower), 0., cal.lower_length * np.sin(lower)])
    upper_end = elbow + R @ np.array([cal.upper_length * np.cos(upper), 0., cal.upper_length * np.sin(upper)])
    # The rigid extension remains horizontal regardless of the upper arm angle.
    axis4 = upper_end + R @ np.array([-cal.horizontal_extension, 0., 0.])
    # Direction decoupled from both arm angles.
    local_dir = np.array([np.sin(pitch) * np.cos(side), np.sin(side), -np.cos(pitch) * np.cos(side)])
    ray = R @ local_dir
    ray /= np.linalg.norm(ray)
    # Servo 4 rotates the shade's eccentric mounting point about its pitch axis.
    # Reference offset is 55 mm to the left. Servo 5 changes ray direction
```

```
    # without moving the mounting point (approximation to be verified).
    # Pitch sign is calibrated from the shade direction: local pitch positive
    # corresponds to the offset rotating toward positive Z.
    axis5 = axis4 + R @ np.array([-cal.shade_axis_offset * np.cos(pitch),
                                  0., cal.shade_axis_offset * np.sin(pitch)])
    bulb = axis5 + cal.bulb_from_axis_5 * ray
    return np.array([origin, shoulder, elbow, upper_end, axis4, axis5, bulb]), ray

def tabletop_hit(q, cal=CAL):
    points, ray = forward(q, cal)
    bulb = points[-1]
    if ray[2] >= -1e-9 or bulb[2] <= 0:
        return None
    distance = -bulb[2] / ray[2]
    return bulb + distance * ray if distance >= 0 else None

def tilt_from_vertical(ray):
    return float(np.degrees(np.arccos(np.clip(-ray[2], -1, 1))))

def _residual(q, target, tilt, q_start, cal):
    points, ray = forward(q, cal)
    bulb = points[-1]
    # Smooth ray/plane residual even when the ray points upward: prevents
    # optimizer from getting stuck at the discontinuity of tabletop_hit().
    dz = min(ray[2], -0.05)
    t = max(0.0, -bulb[2] / dz)
    projected = bulb + t * ray
    xy_error = (projected[:2] - target[:2]) / 100.0
    vertical_penalty = max(0.0, ray[2] + 0.05) * 15.0
    height_penalty = max(0.0, -bulb[2]) / 100.0
    actual_tilt = tilt_from_vertical(ray)
    return np.r_[xy_error * 8., (actual_tilt - tilt) / 15.0,
                 vertical_penalty, height_penalty, (q - q_start) / 100.0 * 0.12]

def inverse(target_xy, tilt, start, cal=CAL, random_starts=10):
    """Constrained numeric IK. Returns (angles, error_mm, tilt_error_deg, hit_or_None).

    The target is NOT guaranteed reachable. Always inspect returned errors.
    """
    target = np.asarray(target_xy, float)
    start = np.clip(np.asarray(start, float), LIMITS[:, 0], LIMITS[:, 1])
    rng = np.random.default_rng(2026)
    seeds = [start, REFERENCE] + [rng.uniform(LIMITS[:, 0], LIMITS[:, 1]) for _ in range(random_starts)]
    best = None
    for seed in seeds:
        result = least_squares(_residual, seed, bounds=(LIMITS[:, 0], LIMITS[:, 1]),
                               args=(target, float(tilt), start, cal), max_nfev=230)
        hit = tabletop_hit(result.x, cal)
        _, ray = forward(result.x, cal)
        error = float(np.linalg.norm(hit[:2] - target[:2])) if hit is not None else float('inf')
        tilt_error = abs(tilt_from_vertical(ray) - tilt)
        score = error + tilt_error * 5 + np.linalg.norm((result.x - start) / 180) * 0.5
        if best is None or score < best[0]:
```

```
            best = (score, result.x, error, tilt_error, hit)
    return best[1], best[2], best[3], best[4]
```